UNITED STATES MILITARY ACADEMY

Share-Out #4 Write-Up

Track A: Functional Image Studio

CDT Owen A. Lang • CS478 Programming Languages • 6 November 2026

Architecture and Semantics

Listing 1: Core types (`Basics.scala`)

```
// An image is a function, not an array. Coordinates are normalized to [0,1].
type Image = (Double, Double) => Color
type Mask = (Double, Double) => Boolean
type Effect = Image => Image
```

A color is a class that consists of three integers, R, G, and B.

Listing 2: `Color` and its blending primitive (`Basics.scala`)

```
final case class Color(r: Int, g: Int, b: Int) {
  require(List(r, g, b).forall(c => c >= 0 && c <= 255))
}

object Color {
  def clip(n: Double): Int =
    math.round(math.max(0.0, math.min(255.0, n))).toInt

  // Provided color blending primitive. Alpha 0 keeps the bottom color.
  def blend(bottom: Color, top: Color, alpha: Double): Color = {
    require(alpha >= 0.0 && alpha <= 1.0)
    Color(
      clip(bottom.r * (1.0 - alpha) + top.r * alpha),
      clip(bottom.g * (1.0 - alpha) + top.g * alpha),
      clip(bottom.b * (1.0 - alpha) + top.b * alpha)
    )
  }
}
```

A layer consists of a name, image, mask, enable flag and opacity.

Listing 3: `Layer` (`Basics.scala`)

```
final case class Layer(
  name: String,
  image: Image,
  mask: Mask,
  opacity: Double = 1.0,
  enabled: Boolean = true
) {
  require(opacity >= 0.0 && opacity <= 1.0)
}
```

Execution trace of composeLayers

Listing 4: `HumanCore.composeLayers` (human-only)

```
def composeLayers(background: Image, layers: List[Layer]): Image =
  (x, y) =>
    layers.filter(x => x.enabled).foldLeft(background(x, y))(
      (acc: Color, item: Layer) =>
        if item.mask(x, y) then
          // do stuff with layer
          Color.blend(acc, item.image(x, y), item.opacity)
        else
          // show past layers
          acc
    )
```

The reasoning for `foldLeft` instead of `fold` or `foldRight` is to satisfy the condition that the layers are added in correct order i.e from left to right. Since `composeLayers` returns an image, we must return a function. Lets follow the execution for the French flag example. Two layers are created to make flag, the white layer and the red layer. The blue layer is considered the background for the compose layers which will end up becoming the base case for the `foldLeft`.

```
var french_flag = HumanCore.composeLayers(blue_base, List(white_layer, red_layer))
```

Listing 5: Layers and masks used to build the French flag (`Preview.scala`)

```
val line_one = StudentWork.halfplane(1, 0, 0.66)   // x < 0.66
val line_two = StudentWork.halfplane(1, 0, 0.33)   // x < 0.33

val red_base   = Given.solid(Color(223, 34, 45))
val white_base = Given.solid(Color(255, 255, 255))
val blue_base  = Given.solid(Color(8, 101, 175))

val red_layer   = Layer("red layer", red_base, line_two)
val white_layer = Layer("white layer", white_base, line_one)
```

First we filter out all disabled layers, then with the result of the filter we call `foldLeft` to start at the furthest left layer in layers, in this case it will be the white layer. The base color that we are working with will be the background color which is blue. The accumulator is the color that has been accumulated from all previous layers while the item is the new layer to be added on top. The first step for determining the new color is to ask if the current layer masks over it or not which is why we call `item.mask` on the current pixel. If the mask returns true then the current pixel is enabled and if it returns false then the pixel is not enabled for the current layer. If the pixel is enabled, we then need to blend its value with all previous values using the previous color, current color in the layer and the opacity of the layer. If the pixel is disabled for the current layer then we pass the accumulator on to the next step. When implemented with the French flag, the two layers each have a mask at the two thirds X and one thirds X respectively to allow for the even bands of vertical colors which compose the French flag. Since each layer overlaps with each other, we do not have to wory about the white covering two thirds since the red one is drawn on top. This is why the order of the layers matters.

Verification

Seven tests were handwritten by me, two were provided by the starter scaffolding, and twenty-nine tests were created by the AI.

Handwritten tests

Listing 6: Handwritten tests (`Checks` in `Preview.scala`)

```
assert(brighten(mirrorHorizontal(invert(checkerboard)))(0.0, 0.0) ==
    ↪ pipeline(checkerboard)(0.0, 0.0),
  "pipeline should match list order")
assert(invert(brighten(mirrorHorizontal(checkerboard)))(0.0, 0.0) !=
    ↪ pipeline(checkerboard)(0.0, 0.0),
  "function order matters")
assert(brightenMax(checkerboard)(0.0, 0.0) == white)
assert(HumanCore.composeLayers(checkerboard, List(left))(0.0, 0.0) == white)
assert(Color.blend(blue, orange, 0) == blue, "Alpha level 0 must keep bottom color")
assert(Color.blend(blue, orange, 1) == orange, "Alpha level 1 must keep top color")
assert(Color.blend(blue, orange, 0.5) == Color(128, 64, 128), "Hand calculation must
    ↪ match function output")
```

The first handwritten test checks if the pipeline is executed in the correct order, from left to right. The second one makes sure that changing the order of the functions is not equal to the pipeline. The third tests if the brighten test has the correct bounds by brightening every channel to 255. The fourth test ensures that halfplane gives the correct mask. The fifth through seventh ensures that the alpha boundaries give the bottom and top color respectively, and the seventh ensures that the function is consistent with the hand calculated values.

Listing 7: Functions under test: `pipeline` and `halfplane` (`StudentWork.scala`)

```
def halfplane(a: Double, b: Double, c: Double): Mask =
  (x, y) =>
    if a * x + b * y < c then
      true
    else
      false

// Compose effects IN LIST ORDER; e.g. List(f,g)(img) == g(f(img)).
def pipeline(effects: List[Effect]): Effect =
  (img: Image) =>
    effects.foldLeft(img)(
      (acc: Image, item: Effect) => item(acc)
    )
```

The AI wrote its own tests to ensure for correct function output and mutant detection, it even includes hand calculated values.

Listing 8: Excerpt of the AI-written mutation-detection test (`AiChecks` in `Preview.scala`)

```
def vignetteOracle(vig: (Double, Double, Double) => Effect): Boolean = {
  val e = vig(0.125, 0.375, 0.5)(solidWhite)
  e(0.5, 0.5) == white && e(0.75, 0.5) == Color(191, 191, 191)
```

```
}
val mutantVignette: (Double, Double, Double) => Effect =
  (inner, outer, strength) => img => (x, y) =>
    val d = math.hypot(x - 0.5, y - 0.5)
    val t = math.max(0.0, math.min(1.0, (d - inner) / (outer - inner)))
    Color.blend(black, img(x, y), strength * t)   // MUTATION: arguments swapped
assert(vignetteOracle((i, o, s) => vignette(i, o, s)), "oracle accepts the real
    ↪ vignette")
assert(!vignetteOracle(mutantVignette), "oracle must detect the swapped-blend
    ↪ mutation")
```

I wrote my tests after finishing my functions and it turned out they all worked first time so nothing meaningful came out of my tests.

AI Extension

The purpose of the AI extension was to see the true extent of what kind of functional effects it could generate. I am not very informed on image generation or image effects so instead of letting Claude choose one, I had Claude implement all the sampled effects and go wild creating sample images. The AI implemented its own tests to check its work. I checked the AI's work by reading through the code and by testing its given functions to ensure everything works the way AI presented. The human only files are `Basics.scala`, `StudentWork.scala`, `HumanCore.scala`, and the `Checks` and `Preview` functions of the `Preview.scala` file.

Listing 9: Representative AI-generated effects (`AiExtension.scala`)

```
def vignette(inner: Double, outer: Double, strength: Double,
             cx: Double = 0.5, cy: Double = 0.5): Effect = {
  require(inner >= 0.0 && outer > inner, "need 0 <= inner < outer")
  require(strength >= 0.0 && strength <= 1.0, "strength must be in [0,1]")
  (img: Image) =>
    (x, y) =>
      val d = math.hypot(x - cx, y - cy)
      val t = clamp01((d - inner) / (outer - inner))
      Color.blend(img(x, y), black, strength * t)
}

def pixelate(columns: Int, rows: Int): Effect = {
  require(columns > 0 && rows > 0, "need at least one column and row")
  (img: Image) =>
    (x, y) => img(cellCenter(x, columns), cellCenter(y, rows))
}

def warp(f: (Double, Double) => (Double, Double)): Effect =
  (img: Image) =>
    (x, y) =>
      val (u, v) = f(x, y)
      img(u, v)

def wave(amplitude: Double, frequency: Double): Effect =
  warp((x, y) => (x + amplitude * math.sin(2 * math.Pi * frequency * y), y))
```

Tradeoffs

One of the tradeoffs you make when choosing function composition is that you have to represent an image with a function, so one has to build an image component by component rather than defining each pixel. With this framework I can't take an existing png and convert it into a function to apply effects on it. One performance trade off is when there are many layers with expensive effects such as blur which runs in $O(n^2)$ each time it is called. Every time an image is rendered, it must run these expensive functions, instead of saving the result for the next effect. One improvement which is plausible is to be able to take in a rendered image and convert it to a function which would solve the expensive effects problem and allow for much more creative art.

Listing 10: Box blur: `samples`^{2} evaluations of the inner image per output pixel (`AiExtension.scala`)

```
def blur(radius: Double, samples: Int): Effect = {
  require(radius >= 0.0, "radius must be non-negative")
  require(samples >= 1, "need at least one sample")
  val offsets = kernelOffsets(radius, samples)
  val kernel = for (dx <- offsets; dy <- offsets) yield (dx, dy)
  (img: Image) =>
    (x, y) => average(kernel.map((dx, dy) => img(x + dx, y + dy)))
}
```

Listing 11: The rendering boundary re-evaluates the whole function for every pixel (`Basics.scala`)

```
def png(image: Image, width: Int, height: Int, filename: String): Unit = {
  require(width > 0 && height > 0)
  val out = new BufferedImage(width, height, BufferedImage.TYPE_INT_RGB)
  for (y <- 0 until height; x <- 0 until width) {
    val c = image((x + 0.5) / width, (y + 0.5) / height)
    out.setRGB(x, y, (c.r << 16) | (c.g << 8) | c.b)
  }
  val file = new File(filename)
  Option(file.getParentFile).foreach(_.mkdirs())
  ImageIO.write(out, "png", file)
}
```